When working with AWS Amplify Gen 2, separating your backend and frontend into independent codebases is a highly effective architecture. It enforces strict microservice isolation, prevents your core backend logic from being tied to a specific UI framework, and allows multiple distinct frontends (e.g., a React web app and a Flutter mobile app) to share the exact same backend infrastructure.
Here is the cleanest strategy to achieve a complete separation of concerns in Gen 2.

The Architecture Structure

Instead of housing everything in a single repository, you will split your system into two distinct codebases:

```
Plaintext
├── 1. backend-repository/      <-- Pure Infrastructure & Business Logic
│   ├── amplify/
│   │   ├── auth/
│   │   ├── data/
│   │   └── backend.ts
│   └── package.json
│
└── 2. frontend-repository/     <-- Pure UI Client (React, Next.js, Flutter, etc.)
    ├── src/
    ├── amplify_outputs.json    <-- Pulled dynamically during CI/CD
    └── package.json
```

Step 1: Set Up the Isolated Backend Repo

Your backend repository doesn't need any frontend files (no HTML, no React components, no Vite configs). It acts strictly as an infrastructure-as-code package.

1. Create a empty directory, initialize it, and scaffold Amplify:
   ```
   Bash
   mkdir core-backend && cd core-backend
   npm init -y
   npm create amplify@latest
   ```

2. Define your Auth, Data schemas, and Lambda functions purely inside the amplify/ folder using TypeScript.
3. Commit this to its own Git repository (e.g., github.com/your-org/core-backend).

Step 2: Set Up the Isolated Frontend Repo

Your frontend repository remains a standard client application without an amplify/ folder. It only needs to know *how to talk* to the backend.

1. Create your frontend app as you normally would (e.g., npm create vite@latest).
2. Install the necessary client-side Amplify libraries:
   Bash
   npm add aws-amplify

3. Commit this to a separate Git repository (e.g., github.com/your-org/web-client).

Step 3: Connect Them via CI/CD (The Essential Link)

The magic link between your separated frontend and backend is the amplify_outputs.json file. This file contains the API endpoints, Cognito User Pool IDs, and region details that the frontend library needs to initialize.
To handle this cleanly across separate deployments, use the following workflows:

A. Local Development (The Sandbox Strategy)

When developing locally, you can run the backend sandbox from your backend repository and direct the configuration file straight into your frontend repository:

Bash
Run this inside your 'core-backend' directory
npx ampx sandbox --outputs-out-dir ../path-to-your-frontend-repo/

This instantly generates and updates the amplify_outputs.json file right inside your frontend client code whenever you modify backend schemas.

B. Production Deployment Workflow

When deploying to staging or production, you configure AWS Amplify Hosting to orchestrate the pipeline automatically.

1. **Deploy the Backend:** Connect your core-backend repository to AWS Amplify. Amplify will deploy your resources (AppSync, Cognito, etc.) and generate a dedicated App ID.
2. **Deploy the Frontend:** Connect your web-client repository to AWS Amplify Hosting.
3. **Inject the Outputs via Build Settings:** In the Amplify Console for your **frontend app**, modify the Build Settings (amplify.yml) to download the backend schema dynamically during the build process before the frontend compiles:

YAML
frontend:

```
phases:
  preBuild:
    commands:
      - npm ci
      # Dynamically pull the latest backend metadata using your backend's App ID and Branch
      - npx ampx generate outputs --app-id <YOUR_BACKEND_APP_ID> --branch main
  build:
    commands:
      - npm run build
artifacts:
  baseDirectory: dist
  files:
    - '**/*'
```

Why This Approach Shines

- **Microservice Isolation:** Your frontend team can commit, test, and deploy UI updates rapidly without risking accidental modifications to cloud resources or Lambda definitions.
- **Function and Schema Reuse:** If you build a mobile application later, you simply point its build settings to the same backend App ID. You instantly reuse the identical database schema, custom Lambda triggers, and authentication profiles without writing a single line of redundant backend code.
- **Clean Environments:** Avoids overcharging a single monolithic deployment block, allowing for granular version control and isolated testing sandboxes for different developer teams.

Are you planning to link multiple client apps to this single backend, or are you focusing on splitting a specific web app?